PrintEase: System Architecture and Trade-offs

Short write-up of project architecture and design trade-offs

PrintEase follows a three-tier, function-oriented architecture built on the MERN stack, decomposed into a User/Admin UI subsystem (React), a Backend Logic subsystem (Node.js/Express), and a Database & Storage subsystem (MongoDB).

The system is further broken down into nine functional processes Authentication -> file upload -> NLP instruction processing -> cost calculation -> payment processing -> print job creation -> queue management -> admin dashboard -> notifications each with its own data flow, as shown in the Level 1 DFD.

This decomposition was a deliberate choice: rather than a monolithic backend, each process owns a narrow responsibility and communicates through well-defined data flows into shared MongoDB collections (users, orders, stored-files), which kept modules independently testable and let different team members own different processes without much coupling.

System Architecture

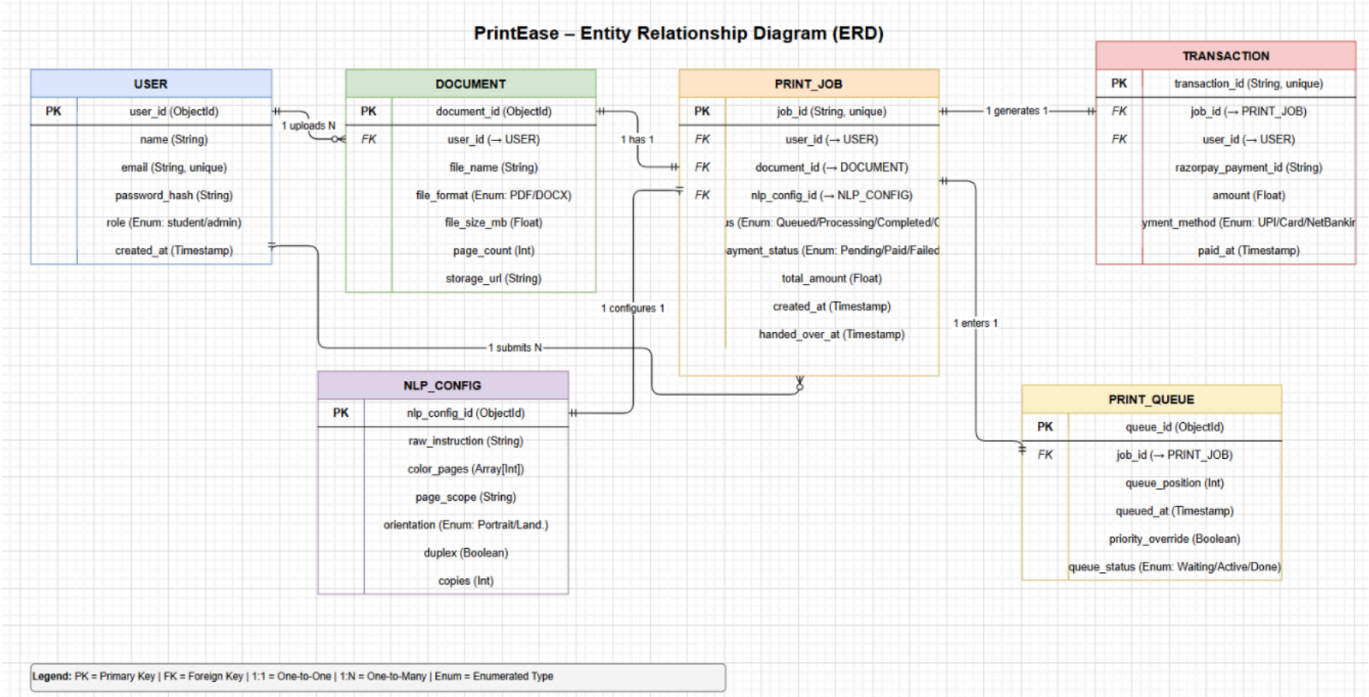
For PrintEase, we decided to use a function-oriented design approach. This choice is driven by the fact that the system's operations can be logically decomposed into distinct subsystems that interact through continuous data transfer and feedback loops. The architecture is partitioned into the following high-level subsystems:

User/Admin UI Subsystem: This serves as the primary interface where students upload documents and provide instructions, and where administrators monitor the active print queue and manage hardware status.

Backend Logic Subsystem: This acts as the central processing hub, responsible for executing cost calculation algorithms, maintaining the FIFO (First-In-First-Out) queue, making calls to the Razorpay payment gateway, and performing all necessary database queries.

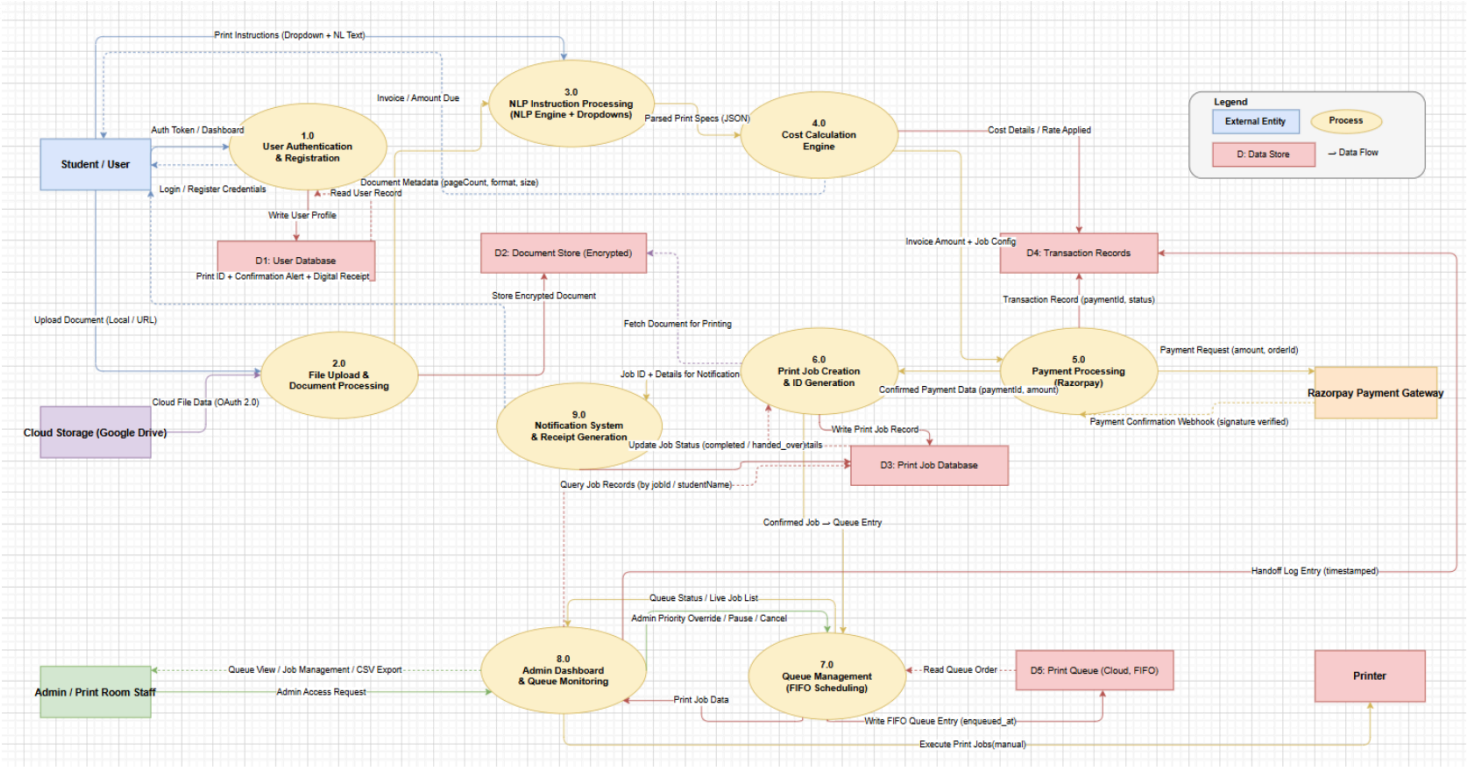
Database & Storage Subsystem: This subsystem provides persistent storage for user profiles and transaction logs while also managing the temporary, encrypted storage of uploaded files until the print job is successfully completed

ER diagram decimating the System Architecture :



Sequence diagram decimating the System Architecture :

DFD showing the internal architecture :



Trade off's

Trade-off 1: Payment-Gated Queue Entry

The most consequential architectural decision is the payment-gated queue. A print job isn't created in the Print Job Database until a Razorpay web hook confirms payment success - visible in the Level 2 DFD for print job generation, where job ID generation is triggered only by confirmed payment data. The trade-off here is added latency and complexity: the system has to sit in a "Pending Payment" state and rely on asynchronous web hook delivery rather than just trusting the client-side payment confirmation. In exchange, the system gains financial integrity - no job can enter the FIFO queue without payment, and no double-charging can occur on retry - which matters more for a system handling real money than shaving a few hundred milliseconds off the checkout flow.

Trade-off 2: Hybrid Dropdown + NLP Instruction Layer

The system supports both structured dropdown input and free-text natural language instructions, with an explicit override protocol - if a user selects "B/W" in the dropdown but writes "print the cover in colour," the NLP-extracted parameter wins. This is more failure-prone and harder to test than a pure-dropdown interface, which is why a dedicated "Final Configuration Summary" confirmation screen was added before payment, to mitigate mis-parsed instructions. The design was chosen because purely dropdown-driven UIs handle simple jobs well and handle nothing else - a student wanting "only page 5 in colour, rest B/W, double-sided" would otherwise need a combinatorially large set of dropdown options. The cost is an added Python NLP service that the Node backend has to bridge into, introducing a second runtime and a network hop (with a 2-second latency budget) purely to support that flexibility.

Trade-off 3: Schema-Level Role Separation

A third trade-off is in the role-based access split baked into the User schema (role: student/admin) rather than maintaining a separate Admin collection or service. This keeps auth simple - one login flow, one JWT issuance path - but means every protected route has to actively check role on top of token validity, and a bug in that check silently becomes a privilege-escalation risk rather than a routing error. Schema-level role was chosen over a separate micro service because the admin surface area (print room staff, single college deployment) was small enough that the

operational overhead of a separate service wasn’t justified.

Trade-off 4: Deferred Hardware Integration

Finally, the system explicitly defers physical printer integration (the actual LAN/IP-based handoff to a network printer) to future scope, as noted in the conclusion. The current system stops at “job ready for pickup” with manual admin execution. This was a scope trade-off rather than a technical one: building and testing against real networked printer hardware was out of reach for a college mini-project timeline, so the system was architected to be printer-agnostic at the boundary - a clean interface point for that integration later - rather than half-building a fragile hardware bridge.